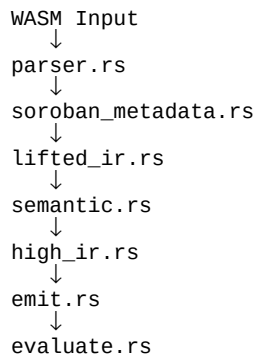


Soroban WASM → Rust/WAT Decompiler Developer Documentation

1. Overview

This project implements a decompiler for Soroban smart contracts. It converts compiled WASM contracts into readable Rust code and annotated WAT. The system is designed as a structured pipeline with clearly separated stages.

2. Pipeline Overview



3. Pipeline Stages

3.1 parser.rs

Parses the WASM module, extracts sections, imports, exports, and function bodies.

3.2 soroban_metadata.rs

Decodes Soroban-specific metadata such as contract functions, types, and environment information.

3.3 lifted_ir.rs

Transforms WASM bytecode into an intermediate representation (IR) with control flow and SSA-like variables.

3.4 semantic.rs

Recovers high-level meaning by mapping low-level operations to Soroban concepts such as storage access and events.

3.5 high_ir.rs

Restructures IR into a higher-level representation similar to an abstract syntax tree (AST).

3.6 emit.rs

Generates final outputs: reconstructed Rust code and annotated WAT.

3.7 evaluate.rs

Compares generated Rust with original source using AST similarity metrics.

4. Core Files

- 1 parser.rs → WASM parsing
- 2 soroban_metadata.rs → metadata decoding
- 3 lifted_ir.rs → IR generation
- 4 semantic.rs → semantic recovery
- 5 high_ir.rs → high-level IR

- 6 emit.rs → Rust and WAT generation
- 7 evaluate.rs → accuracy evaluation
- 8 pipeline.rs → orchestrates all stages

5. CLI Usage

The CLI entry point runs the pipeline on a given WASM file and produces outputs in the 'out/' directory.

6. Outputs

- 1 reconstructed.rs → Decompiled Rust code
- 2 annotated.wat → Readable WASM
- 3 report.json → Accuracy results

7. Known Limitations

- 1 Semantic recovery may be incomplete in some cases
- 2 Fallback placeholders may still appear
- 3 Reconstruction accuracy is not fully consistent across all inputs